

NETRA Backend Architecture — Q&A; Reference Document

Version: 1.1 — Updated 2026-09-03 (Deterministic Random Forest ML + Rule Engine architecture)

Codebase Ref: /Users/iamspars00321/Desktop/newantigravworkfolder/netra/backend

Status key: ■ Implemented | ■■ Partial | ■ We haven't decided this yet.

Domain 1 — Catalog Auto-Population

Q1. When a user uploads any file through the web Forensic Detection Sandbox, does it automatically get added to the threat catalog?

Yes — this is a confirmed product decision. Every video, image, audio, or text submission through the sandbox that returns a positive/suspicious verdict should be auto-indexed into the `threat_catalog` table. Currently, the text path via the Public API (`public_api.py` line 89–120) auto-inserts on detection. The video/image paths do not yet auto-insert from the sandbox UI — they rely on the GPU worker (video) or explicit `/report` calls.

Q2. Which code path triggers the auto-insert into `threat_catalog` for text submissions?

`backend/api/routes/public_api.py` lines 89–120. `insert_threat_item()` from `db.py` is called whenever `scam_detected == True`.

Q3. Which code path triggers auto-insert for image submissions (OCR path)?

■ We haven't decided this yet. The OCR pipeline (`ocr_scam_pipeline.py`) returns a verdict dict to the frontend but does NOT currently call `insert_threat_item()`. Whether the web sandbox should auto-insert on image scan is undecided.

Q4. Which code path triggers auto-insert for video submissions?

■ We haven't decided this yet. The GPU worker (SQS consumer) is responsible for writing results to DynamoDB jobs table. Whether the worker also calls `insert_threat_item()` after analysis is complete is not yet implemented.

Q5. Is there a deduplication check before inserting into the catalog?

Partial. `db.py` uses `INSERT OR REPLACE INTO threat_catalog` (line 198), which deduplicates by `id` (primary key). However, two submissions of the same content with different generated IDs will both be inserted as separate records. Content-hash deduplication is not implemented.

Q6. Who can manually add items to the threat catalog without going through the sandbox?

Via POST `/api/v1/threat-intelligence/report` (authenticated). Via the Public API POST `/api/v1/public/detect/scam-text` (API-key required, auto-inserts). Via Telegram/WhatsApp bot submissions (see Domain 17).

Domain 2 — Video Pipeline

Q7. What happens step-by-step when a user uploads a video?

- Frontend POSTs to `POST /api/v1/detect/full` (`detect.py` line 20)
- File is validated for MIME type (mp4/mov/webm/avi) and size (≤ 100 MB)
- File is streamed to S3 bucket `netra-media-uploads` at key `{job_id}/input.mp4`
- A job record is written to DynamoDB table `netra-jobs` with status `queued`
- A message is sent to SQS with `{job_id, s3_key, created_at}`
- API returns `{job_id, status: "queued"}` immediately
- Frontend polls `GET /api/v1/jobs/{job_id}` every 2 seconds

Q8. What content types are accepted for video upload?

`video/mp4`, `video/quicktime`, `video/webm`, `video/avi`, `video/x-msvideo` — see `detect.py` line 17.

Q9. What is the maximum video file size?

100 MB — enforced at `detect.py` line 11 (`MAX_FILE_SIZE_MB = 100`).

Q10. What does the GPU worker do with the video after picking it from SQS?

■ We haven't decided this yet. The SQS worker/consumer (the GPU-side service that reads SQS messages, downloads from S3, runs the ML pipeline, and writes results to DynamoDB) does not exist in the current codebase.

Q11. What ML models run on a video?

The pipeline stack (defined locally, not yet wired to the worker) includes:

- `detectors/spatial.py` — SBI spatial face manipulation detector
- `detectors/audio.py` — MelodyMachine Wav2Vec2 audio deepfake detector
- `detectors/clip_probe.py` — CLIP zero-shot probe
- `gend_engine.py` — GenD ViT-L/14 (WACV 2026) foundation deepfake detector
- `fusion.py` — Gated weighted fusion (60% GenD + 25% Spatial + 15% CLIP; audio gated)

Q12. How does the frontend know when the video analysis is complete?

Polling via `GET /api/v1/jobs/{job_id}` (implemented in `jobs.py` lines 27–64). Frontend polls every 2 seconds until `status == "complete" or "error"`.

Q13. Where is the video analysis result stored?

In the DynamoDB `netra-jobs` table as a JSON string in the `result` field.

Q14. How long is the video stored in S3 after analysis?

■ We haven't decided this yet. No S3 lifecycle policy or TTL is configured.

Q15. Can users download or stream their uploaded video back?

Yes — `GET /api/v1/jobs/{job_id}/video-url` generates a 1-hour presigned S3 URL (`jobs.py` lines 102–120).

Q16. Can users download a PDF report of their video analysis?

No. `GET /api/v1/jobs/{job_id}/report.pdf` returns HTTP 501 — "PDF report generation coming in Phase 7."

Domain 3 — Audio Pipeline

Q17. Does NETRA accept standalone audio file uploads (MP3, WAV, M4A)?

■ We haven't decided this yet. Currently `detect.py` only accepts video MIME types. Standalone audio upload is not implemented.

Q18. How is audio currently handled in the video pipeline?

The `extractor.py` module extracts 16kHz mono WAV audio from uploaded video using FFmpeg. This extracted WAV is passed to `detectors/audio.py` (`AudioDeepfakeDetector`).

Q19. What models run on audio?

Primary: `MelodyMachine/Deepfake-audio-detection-V2` (`Wav2Vec2` + `AASIST`, ~99.7% accuracy on `ASVspoof`). Fallback: `facebook/wav2vec2-base-960h` with mismatched-size 2-class head.

Q20. What happens if a video has no audio track (silent video)?

The fusion engine detects `audio_score < 0.1` and sets `audio_weight = 0.0`, removing audio from the fusion entirely (`fusion.py` lines 58–70).

Q21. What audio file formats would be supported if standalone audio upload is added?

■ We haven't decided this yet.

Q22. If a standalone audio file is uploaded, does it get auto-inserted into the threat catalog?

■ We haven't decided this yet.

Domain 4 — Image Pipeline

Q23. What happens when a user uploads an image in the Forensic Detection Sandbox?

- Frontend POSTs to `POST /api/v1/detect/image-ocr` (`detect.py` line 86)
- Image is validated (jpeg/png/webp/bmp, ≤50 MB)
- `run_image_ocr_and_scam_detection()` is called synchronously
- Result is returned immediately — no job queue, no polling

Q24. What OCR engines are tried and in what order?

- `PaddleOCR v2.7` (primary)
- `EasyOCR` (fallback — CPU, English)
- `PyTesseract` / `Tesseract OCR` (last resort)

All lazy-loaded on first use.

Q25. What happens if the image contains no text?

OCR returns empty `full_text`. Pipeline returns `risk_score: 5, scam_type: "CLEAN_IMAGE", reason: "No text detected in image."` Frontend displays "No text found."

Q26. What IOCs are extracted from image text?

Indian mobile numbers, UPI IDs (@okaxis/@paytm/etc), HTTP/HTTPS URLs, and .apk filenames via regex (`ocr_scam_pipeline.py` lines 43–55).

Q27. Do APK or UPI IOCs in an image auto-escalate the risk score?

Yes. APK found → `risk_score >= 92, is_scam = True`. UPI + payment keywords → `risk_score >= 88` (`ocr_scam_pipeline.py` lines 170–177).

Q28. Does the image pipeline run visual deepfake detection (not just OCR)?

Only via the Public API POST `/api/v1/public/detect/image` (API-key required). The web sandbox image path currently only does OCR.

Q29. After an image scan, does the result get saved to the threat catalog?

■ We haven't decided this yet. The `/detect/image-ocr` endpoint does not call `insert_threat_item()`.

Domain 5 — Text Pipeline

Q30. What happens when a user submits text to the Scam Checker?

- Frontend POSTs to POST `/api/v1/detect/scam` (`scam.py`)
- Text is validated (non-empty, length ≥ 5 characters)
- Deterministic Random Forest ML + heuristic pattern rule matrix runs synchronously
- `ScamResponse` returned immediately in <15ms

Q31. How does the scam detection pipeline work without LLMs?

Stage 1: TF-IDF vectorization transformed through a trained Random Forest model (`scam_rf_model.pkl`) to compute a base statistical threat probability (0–100%).

Stage 2: Deterministic heuristic rule matching scans for 6 high-risk Indian cyber fraud typologies (Digital Arrest, APK Malware, Electricity KYC, Stock Trading Fraud, Banking UPI Phishing, Job Scam).

High-confidence rule matches dynamically calibrate the final score and synthesize structured forensic reasons.

Q32. Are any Large Language Models (LLMs) used in scam text analysis?

No. LLMs have been completely removed from NETRA. The system uses only deterministic Random Forest ML and heuristic rule matrices for speed, privacy, and zero API costs.

Q33. Does the text detection pipeline require external API keys?

No. The entire text pipeline runs fully locally and on-premises using the local pickle models and Python regex rules. No third-party API keys or cloud AI services are called.

Q34. After text is scanned via web sandbox, does it get auto-inserted into the catalog?

No. Only the Public API endpoint auto-inserts. Web sandbox /detect/scam does not. Inconsistency to be resolved.

Q35. What scam verdict labels are used?

CRITICAL – Almost Certainly a Scam (≥ 70), HIGH RISK – Likely Scam (40–69), CAUTION – Suspicious Patterns Found (< 40 , is_scam=True), CAUTION – Low Risk / Inconclusive (40+, is_scam=False), SAFE – No Suspicious Patterns.

Domain 6 — Threat Catalog

Q36. What is the schema of the threat_catalog table?

id (PK), title, type (video_deepfake/image_deepfake/scam_text/audio_clone), threat_category, source_platform, fake_probability (REAL 0.0–1.0), verdict, risk_level, thumbnail_url, media_url, lat, lng, city, state, country, location_source, device_model, software_used, extracted_iocs (JSON), fir_dossier (JSON), upvotes_count, created_at.

Q37. What are the valid values for threat_category?

DIGITAL_ARREST, ELECTRICITY_KYC, STOCK_FRAUD, JOB_SCAM, VOICE_CLONE, IMPERSONATION, DEEPFAKE_IMPERSONATION, INVESTMENT_FRAUD, APK_TROJAN, BANKING_PHISHING.

Q38. What is fir_dossier and what does it contain?

JSON with incident_summary, applicable_laws (e.g., IT Act Section 66D, BNS Section 318(4)), and recommended_action. Written at ingestion time for API/text submissions. FIR generation for video/image is not yet implemented.

Q39. How does upvoting work?

POST /api/v1/threat-intelligence/{id}/upvote increments upvotes_count by 1. No per-user deduplication — unlimited upvotes from any client.

Q40. Is there a per-user upvote deduplication?

■ We haven't decided this yet.

Q41. Can catalog entries be deleted or edited?

No. Read + insert + upvote only. No delete or edit endpoints exist.

Domain 7 — Geolocation

Q42. How does the system determine lat/lng/city/state of a threat entry?

Priority order:

- EXIF GPS data from image/video metadata

- User-reported city matched against INDIAN_METROS lookup (8 cities)
- Fallback to New Delhi (28.6139, 77.2090)

Q43. What does `location_source` mean?

EXACT_GPS — real EXIF GPS tags. ESTIMATED_TELECOM — city-level estimate. REGIONAL_HOTSPOT — default metro fallback.

Q44. Does the system do reverse geocoding?

- We haven't decided this yet. No external geocoding API is integrated.

Q45. Is geolocation limited to India?

Yes. INDIAN_METROS contains only 8 Indian cities. `country` defaults to "India".

Q46. Does the system extract geolocation from video metadata?

Yes — `exif_engine.py` uses `ffprobe` subprocess for video container metadata GPS extraction.

Domain 8 — Job System

Q47. What is the DynamoDB job record schema?

Fields: `job_id` (S), `status` (S: `queued/processing/complete/error`), `progress` (N: 0–100), `current_stage` (S), `s3_key` (S), `created_at` (S), `file_size_mb` (N). Worker adds `result` (S: JSON string) on completion.

Q48. How does the frontend poll for job status?

`GET /api/v1/jobs/{job_id}` reads DynamoDB directly. Frontend polls every 2 seconds until `complete/error`.

Q49. Is there a WebSocket progress push?

Partially. `WS /ws/{job_id}` exists and polls DynamoDB every 2s pushing updates, but is not actively used by the frontend yet.

Q50. What are the defined job progress stages?

- We haven't decided this yet. No standard spec for what stage strings the worker should write to DynamoDB.

Q51. What happens if a job fails mid-processing?

- We haven't decided this yet. No DLQ, no retry logic, no `error` status writer in the current codebase.

Q52. Is there a TTL or cleanup on DynamoDB job records?

- We haven't decided this yet.

Domain 9 — API Keys & Auth

Q53. How are API keys issued?

`create_api_key()` in `db.py` generates `sk_live_` prefixed token, SHA-256 hashes it, stores hash only. Raw token returned once, never stored.

Q54. How are API keys verified?

`verify_api_key` FastAPI Depends reads X-API-Key header, hashes it, looks up in `api_keys` table, checks quota, increments `used_requests`.

Q55. What happens when quota is exceeded?

`verify_and_consume_key()` returns `{"error": "QUOTA_EXCEEDED"}`. Request is rejected.

Q56. Is there per-minute rate limiting?

■ We haven't decided this yet. Monthly count quota only.

Q57. How does an admin create/delete API keys?

Via undocumented endpoints in `threat_intel.py`. No separate admin auth.

Q58. Do web sandbox endpoints require an API key?

No. `/detect/scam`, `/detect/full`, `/detect/image-ocr` are unauthenticated. Only `/api/v1/public/*` requires auth.

Domain 10 — News Intelligence

Q59. How does NETRA get cyber scam news?

Background daemon thread every 24h calls Tavily API with query about Indian cybercrime/deepfake news.

Q60. What if TAVILY_API_KEY is not set?

Skips live crawl. Returns curated `CURATED_SCAM_NEWS` hardcoded articles (6 pre-written articles as seed).

Q61. Where is news stored?

Separate SQLite file `threat_catalog.db` (different from main `netra.db`). `scam_news` table.

Q62. How is category assigned to Tavily-crawled articles?

Keyword matching: "arrest" → `DIGITAL_ARREST`, "deepfake" → `DEEPFAKE_IMPERSONATION`, else → `INVESTMENT_FRAUD`. Risk hardcoded to `CRITICAL`.

Q63. Is there deduplication for news?

Partial. `INSERT OR REPLACE` by id. Tavily-crawled IDs include timestamp — same article re-crawled gets a new ID.

Q64. Can users trigger a manual news refresh?

Yes — `POST /api/v1/news/refresh` triggers immediate crawl.

Domain 11 — Community Posts

Q65. What authentication is required to publish a post?

Google OAuth via frontend-only `GoogleAuthModal`. Backend does NOT verify the auth token — any caller can POST to `/api/v1/community/posts`.

Q66. Is there moderation or abuse prevention?

■ We haven't decided this yet. No profanity filter, rate limiting on posts, or admin review queue.

Q67. How is read time calculated?

`word_count / 200` words per minute, minimum 1 minute.

Q68. Can posts be edited or deleted?

■ We haven't decided this yet. No edit/delete endpoints exist.

Q69. What post categories exist?

DEEPFAKE, SCAM_ANALYSIS, VOICE_CLONE, SAFETY_GUIDE, THREAT_INTEL.

Domain 12 — ML Models

Q70. What are the primary forensic ML systems?

- GenD ViT-L/14 — visual deepfake (foundation model, WACV 2026)
- AudioDeepfakeDetector — Wav2Vec2-based voice clone / synthetic audio detection
- CLIP probe — zero-shot visual semantic probe
- Random Forest + TF-IDF — statistical text scam classifier

Q71. Where are Random Forest models stored?

`backend/netra/pipeline/models/scam_rf_model.pkl + tfidf_vectorizer.pkl` (loaded via `joblib`).

Q72. How is GenD ViT-L/14 loaded?

Lazy-loaded on first inference via `AutoModelForImageClassification.from_pretrained("yermandy/GenD_CLIP_L_14")`. Falls back to local hypersphere simulator if HF download fails.

Q73. Does GenD run on GPU or CPU?

Auto-detects CUDA → MPS (Apple Silicon) → CPU. Production target: CUDA on AWS g4dn/g5.

Q74. What is the model fusion weight distribution?

All available: 60% GenD + 25% Spatial SBI + 15% CLIP. Audio: 40% of final when `audio_score >= 0.3`, else 0%.

Q75. What hardware runs the API server vs ML models?

API server: lightweight t3.micro (CPU only — explicitly stated in `detect.py` comment). ML models: separate GPU worker on g4dn/g5.

Domain 13 — Text Heuristics & Deterministic Reasoning Engine

Q76. Does NETRA rely on generative cloud models or external text APIs?

No. All text analysis is performed deterministically on-premises using local Random Forest ML and specialized rule-based pattern matching.

Q77. What are the advantages of this deterministic approach?

Sub-15ms execution latency, zero third-party API spend, 100% predictable classifications, and complete data privacy without streaming user data to external cloud services.

Q78. How does the rule-based reasoning engine generate scam explanations?

When a scam message is detected, the regex pattern engine matches against specific Indian cybercrime typologies (e.g. Digital Arrest keywords, unverified APK installers, fake electricity disconnection notices, illegal stock tips) and generates clear, human-readable explanations specifying the exact indicators found.

Q79. What happens if the Random Forest model file is missing or corrupted?

The engine catches the loading exception and falls back to the deterministic regex pattern rule matrix alone, which still detects high-risk scam patterns with 70–92% confidence.

Q80. Does NETRA require any paid third-party AI keys for text analysis?

No. Text analysis is 100% self-hosted, offline-capable, and requires zero paid AI subscriptions.

Domain 14 — Database

Q81. What database engine is used?

SQLite with WAL mode, `busy_timeout=5000ms`. Two separate files: `api/netra.db` (main catalog, keys, community) and `threat_catalog.db` (news articles).

Q82. Is SQLite suitable for production?

■ We haven't decided this yet. SQLite with WAL handles moderate reads well but write-heavy or multi-server deployments require PostgreSQL. Migration not planned.

Q83. Is there a database migration system?

No. `CREATE TABLE IF NOT EXISTS` on startup only. Schema changes require manual SQL.

Q84. Are there database backups?

■ We haven't decided this yet.

Q85. What indexes exist?

`threat_catalog`: indexes on `threat_category`, `type`, `created_at`, `city`.

`community_posts`: indexes on `category`, `created_at`.

Domain 15 — Scalability

Q86. What is the current write bottleneck?

SQLite — no concurrent writes from multiple API server processes. GPU inference single-threaded per worker.

Q87. How many concurrent video jobs can the system handle?

■ We haven't decided this yet. One SQS queue + one GPU worker = serial processing. Horizontal scaling requires multiple SQS consumer workers.

Q88. Can the API server be horizontally scaled?

Partially. FastAPI is stateless if SQLite is replaced with PostgreSQL/DynamoDB. Multiple replicas behind a load balancer would work then.

Q89. Is there a caching layer?

■ We haven't decided this yet. No Redis or Memcached.

Domain 16 — Security

Q90. What is the current CORS configuration?

`allow_origins=["*"]` — fully open (`server.py`). Must be restricted to frontend domain in production.

Q91. Is there file upload input validation?

MIME type header check + file size limit. No magic-byte validation — MIME can be spoofed.

Q92. Is there SQL injection protection?

Yes — all queries use parameterized `?` placeholders throughout `db.py`.

Q93. Are there unauthenticated write endpoints?

Yes — `POST /api/v1/threat-intelligence/report` allows anyone to insert into the threat catalog. Intentional (crowdsourcing) but no abuse prevention.

Q94. Is uploaded media scanned for malware?

■ We haven't decided this yet. Files streamed to S3 without antivirus scanning.

Domain 17 — Telegram / WhatsApp Bots

Q95. Do the bots exist?

Partially. Route files `telegram_webhook.py` and `whatsapp_webhook.py` exist. `telegram_bot.py` service exists. Bot was running as daemon in previous sessions.

Q96. How do bot-submitted media enter the threat catalog?

■ We haven't decided this yet. Full pipeline from bot submission → threat catalog insertion is not wired.

Q97. What bot commands are supported?

`/start`, `/help`, `/status` , and sending video/YouTube URLs for analysis.

Domain 18 — Public API

Q98. What Public API endpoints exist?

- POST /api/v1/public/detect/scam-text — text scam analysis + auto-catalog insertion
- POST /api/v1/public/detect/image — GenD deepfake + EXIF metadata

All require X-API-Key header.

Q99. What API tiers and quotas exist?

Tiers: free (100/month default), developer, pro. No per-tier enforcement logic beyond the quota integer.

Q100. Is the API versioned?

All routes use /api/v1/ prefix. No breaking-change versioning strategy.

Q101. Is there auto-generated API documentation?

Yes — FastAPI generates OpenAPI docs at /docs (Swagger UI) and /redoc.

Domain 19 — Error Handling

Q102. What happens if S3 upload fails?

HTTP 500 raised. No retry. S3 file may have been partially uploaded with no cleanup.

Q103. What happens if SQS send fails?

HTTP 500 raised. The S3 file was already uploaded but no job was queued.

Q104. Is there a dead-letter queue for failed SQS messages?

■ We haven't decided this yet.

Q105. What happens if DynamoDB is unavailable during polling?

GET /jobs/{job_id} raises HTTP 500 with DynamoDB error message.

Q106. What happens if all OCR engines fail on an image?

Returns empty full_text → "No text detected" clean verdict path. Errors logged as warnings per engine.

Domain 20 — Deployment

Q107. What AWS services are used?

S3 (video storage), SQS (job dispatch), DynamoDB (job tracking), EC2 (t3.micro for API server, g4dn/g5 for GPU worker).

Q108. What environment variables are required?

S3_BUCKET_MEDIA, SQS_QUEUE_URL, DYNAMO_TABLE_JOBS, AWS_DEFAULT_REGION,
TAVILY_API_KEY, NETRA_DB_PATH, HF_TOKEN.

Q109. Is there a CI/CD pipeline?

■ We haven't decided this yet. No CI/CD config files in the repository.

Q110. How is the backend started?

uvicorn api.server:app --host 0.0.0.0 --port 8000 --reload from backend/ with
PYTHONPATH set.

Q111. Is there a Dockerfile?

■ We haven't decided this yet.

Q112. Does the GPU worker binary exist?

■ We haven't decided this yet. The SQS consumer worker is not in the repository.

Summary Table — Implementation Status

All answers sourced directly from reading the NETRA backend codebase.